Abstractions and logic

This is the canonical index of code representations. The file name `irs.typ` is historical; the canonical term is **abstraction**. The current code names are shown beside each canonical name so the documents and the source can be read together.

Canonical names

Generated from `docs/architecture.edn` by `tools/architecture-check.fsx`.

Canonical	Code	Role
`ExpressionLogic`	`ExpressionAst.Expression`	Documentation: `docs/general-ir.typ` General expression abstraction: literals, words, calls, partials, holes, pipelines, quotations, blocks, let, lambda, if, match, declarations, conditions, restarts, generics, messages.
`PatternLogic`	`PatternAst.Pattern`	Documentation: `docs/general-ir.typ` Pattern abstraction: wildcard, role-carrying variables, literals, constructors, typed patterns; templates elaborate to closed patterns.
`PackageLogic`	`PackageAst.PackageDocument`	Documentation: `docs/general-ir.typ` Package and unit abstraction:

Canonical	Code	Role
		Documentation identity, source hash, imports, exports, capabilities, kinds, predicates, ordered units.
`SymbolicAbstraction`	`TermIr.Term`	`docs/term-ir.typ` Interpreter's symbolic term list and quotation body: literal, word, quote, branch, slot, closure, fail.
`LoweredAbstraction`	`LoweredInstructions.Instruction` + `LoweredAbstraction`	`docs/native-dotnet-port.typ` Lowered instruction abstraction and VM: literals, slots, calls, closures, tensor operations, branches, failure.
`TypeRetrieval`	`TypeInference`	`docs/type-inference.typ` Retrieves and elaborates types against a signature table; not

Canonical	Code	Documentation
		Role: a type checker.
`Portable documents`	`TermProgram, ExpressionDocument, PatternDocument, PackageDocument`	Canonical schema-1 JSON forms used at the image, ABI, persistence, and package boundaries. Docs/general-ir.typ
`DomainExpression and SMT`	`DomainExpression, Smt`	Verification abstraction: typed domain expressions, sorts, guards, obligations, and SMT encodings. Verification/type-lattice.typ
`Image values`	`Tensor, EntryIdentity, Store, Lexicon`	Runtime value model; not code, but the representation every abstraction lowers into. Docs/memory-model.typ
`Execution state`	`KernelState, EvaluationState`	State, not code: store, stack, program, roots, Docs/evaluator-slicing-and-scheduler.typ

Canonical	Code	Role / Documentation
		frames, scheduler, and effects.

Everything is an implementation of some logic

An abstraction is an implementation of a logic. This is the teaching frame:

- the tests implement **test logic**
- expression trees implement **expression logic**
- patterns implement **pattern logic**
- a decision table implements **decision logic**
- an SMT obligation implements **formal logic**
- the kernel implements the **lowered abstraction**
- the F# Prelude implements the **symbolic abstraction**.

The project does not need to rename every implementation to "logic". The canonical names above are the ones used in the documents; the code names remain the implementation detail until a rename is scheduled. This is the framing that lets a child see a function, a test, a table, and a proof as the same kind of object: a piece of logic with an implementation.

Dependency graph

Generated from `docs/architecture.edn`.

From	Kind	To	Rationale
`expression-logic`	`embeds`	`pattern-logic`	PatternMatch clauses carry PatternLogic patterns.
`expression-logic`	`lowers-to`	`lowered-abstraction`	EvaluatorLowering maps the expression subset to instructions.
`package-logic`	`contains`	`expression-logic`	A PackageUnit root is an ExpressionLogic node.
`expression-logic`	`serializes-to`	`portable-documents`	ExpressionPortable writes the canonical expression document.
`pattern-logic`	`serializes-to`	`portable-documents`	PatternPortable writes the canonical pattern document.
`package-logic`	`serializes-to`	`portable-documents`	PackagePortable writes the canonical package document.
`expression-logic`	`inferred-by`	`type-retrieval`	TypeInference elaborates against the signature table.
`lowered-abstraction`	`executes-on`	`image-values`	The VM reads and writes the image store.
`symbolic-abstraction`	`executes-on`	`image-values`	The interpreter reads and writes the image store.

From	Kind	To	Rationale
`domain-expression`	`projects-to`	`image-values`	Verification expressions project into the image value model.

```
expression-logic -> pattern-logic : embeds
expression-logic -> lowered-abstraction : lowers-to
package-logic -> expression-logic : contains
expression-logic -> portable-documents : serializes-to
pattern-logic -> portable-documents : serializes-to
package-logic -> portable-documents : serializes-to
expression-logic -> type-retrieval : inferred-by
lowered-abstraction -> image-values : executes-on
symbolic-abstraction -> image-values : executes-on
domain-expression -> image-values : projects-to
```

Decision tables: where the source comes from

A decision table is not synthesized from the prose specification in the current slice. Its source is a **table document**:

```
table jump-or-run
policy: unique
when player has health at least 2, every player can jump.
end
```

`DecisionTableSurface.parse` reads that document, tokenizes each row as a Basic English transition, and turns each row's conditions into guards and its verb into the outcome. The host supplies the domain's field getters and the outcome body (`binding`); `DecisionTable` then selects rows and checks coverage and disjointness. The demo's table text in `bases/scale-cli/entry.fs` is a hand-written example; the product path is the parsed table document, not a hard-coded F# table.

So the table is an abstraction of decision logic: its source is the table text, its domain comes from the host, and its two properties are checked by the solver. The table is authored or generated; it is not inferred from the prose sentences.

The `verification` directory

The canonical target is one unified `verification` node:

```
verification/
  formal/   Z3, the refiner, diagnostics, and decision tables
  tests/    the example, property, granular, and differential suites
  typing/   type checking (a later layer)
```

The existing code is not moved yet: the formal layer is `components/verification`, the tests are `development/scale`, and type retrieval is `components/typing`. The target is documented now so the move is a mechanical step, not a new design. Type retrieval stays separate from type checking.

Recommendations

- **One canonical abstraction per layer.** The surface produces `ExpressionLogic`; execution consumes `LoweredAbstraction`; portability uses the canonical JSON document for that layer. Do not add a second format for the same layer.
- **Use `PackageLogic` for books/spells.** A book is a package with exports; a spell is a package with `main`. Do not invent a second module format.
- **Keep `PatternLogic` stable.** It is shared by expression matches, pattern analysis, and verification.

- **Resolve `SymbolicAbstraction` versus `LoweredAbstraction`.** The first is the interpreter/quotation form; the second is the compiled form. Keep the split explicit, or retire the symbolic form from execution and keep it as a portable quotation document.
- **Pin the schemas.** Every portable document carries schema 1 and a round-trip/property test.
- **Label stability.** In every abstraction document, state whether the type is a surface, a portable contract, or an internal optimization.